

Guia Administrativo

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [administrative-guide.pt-br.pdf](#)

Este guia documenta a infraestrutura de CI/CD, os GitHub Workflows, os ambientes protegidos, os segredos, as variáveis, as permissões e o processo de release do repositório `awslabs/aidlc-workflows`.

Público-alvo: Administradores de repositório, mantenedores e agentes de codificação com IA que trabalham neste repositório.

Documentação relacionada:

- [Guia do Desenvolvedor](#) — Execução de builds localmente (CodeBuild + `act`)
- [Diretrizes de Contribuição](#) — Processo de contribuição e convenções
- [README](#) — Configuração e uso para o usuário final

Índice

- [Visão Geral do Repositório](#)
- [Arquitetura de CI/CD](#)
- [Referência de Workflows](#)
- [Workflow de Release PR](#)
- [Workflow de Tag Release](#)
- [Workflow do CodeBuild](#)
- [Workflow de Release](#)
- [Workflow de Validação de Pull Request](#)
- [Workflow de Scanners de Segurança](#)
- [Ambientes Protegidos](#)
- [Segredos e Variáveis](#)
- [Modelo de Permissões](#)

- [Postura de Segurança](#)
 - [Requisitos para Achados de Segurança](#)
 - [Propriedade de Código](#)
 - [Processo de Release](#)
 - [Configuração do Changelog](#)
 - [Atualizando Versões Fixadas](#)
-

Visão Geral do Repositório

Este repositório publica a metodologia **AI-DLC (AI-Driven Development Life Cycle)** como um conjunto de arquivos de regras markdown em `aidlc-rules/`. A infraestrutura de CI/CD gerencia:

- **Integração contínua** via AWS CodeBuild (avaliação e relatórios)
- **Distribuição de releases** via GitHub Releases (arquivos de regras compactados)
- **Geração de changelog** via git-cliff (changelog-first: atualizado antes do release, incluído no commit com tag)

```
aws-labs/aidlc-workflows/
├── .github/
│   ├── CODEOWNERS
│   ├── ISSUE_TEMPLATE/           # Modelos de bug, feature, RFC, docs
│   ├── labeler.yml               # Regras de auto-label (mapeamento caminho → label)
│   ├── pull_request_template.md  # Modelo de PR com declaração do contribuidor
│   └── workflows/
│       ├── codebuild.yml         # CI via AWS CodeBuild
│       ├── pull-request-lint.yml  # Validação de PR (título, labels, gates de merge)
│       ├── release.yml           # GitHub Release no push de tag
│       ├── release-pr.yml        # PR de Changelog antes do release
│       ├── security-scanners.yml  # Suite de varredura de segurança (6 scanners)
│       └── tag-on-merge.yml       # Auto-tag no merge do release PR
├── .claude/
│   └── settings.json             # Configurações compartilhadas do projeto Claude Code
├── aidlc-rules/                  # O produto distribuível
│   ├── aws-aidlc-rules/          # Regras de workflow principais
│   └── aws-aidlc-rule-details/    # Regras detalhadas por fase
├── cliff.toml                    # Configuração do changelog com git-cliff
├── docs/
│   ├── ADMINISTRATIVE_GUIDE.md   # Este arquivo
│   └── DEVELOPERS_GUIDE.md       # Instruções de build local
└── scripts/
    └── aidlc-evaluator/           # Framework de avaliação (em desenvolvimento)
```

Arquitetura de CI/CD

Seis workflows formam dois pipelines distintos, uma suite de varredura de segurança e um gate de validação de pull request:

Pipeline 1: Release (changelog-first)

```
flowchart TD
    A["workflow_dispatch\n(input de versão opcional)"] --> B["release-pr.yml"]
    B --> C["Determina versão\n(input ou git-cliff)"]
    C --> D["Gera CHANGELOG.md\ncom git-cliff"]
    D --> E["Abre PR: release/vX.Y.Z\ncom CHANGELOG atualizado"]

    E --> F["Humano revisa\ne faz merge do PR"]

    F --> G["tag-on-merge.yml"]
    G --> H["Extraí versão do\nnome do branch"]
    H --> I["Cria tag vX.Y.Z\nno SHA do commit de merge"]

    I --> J["Dispara release.yml"]
    J --> K["release.yml\ncria draft release\ncom zip das regras"]
    K --> L["Dispara codebuild.yml\napós existir o draft"]
    L --> M["Aprovação manual\n(ambiente codebuild)"]
    M --> N["Executa AWS CodeBuild\nenvia artefatos ao draft"]

    K --> O["Humano revisa\ne publica o draft"]
    N --> O

    P["workflow_dispatch\n(seleciona tag na UI)"] -.->|"disparo manual\nde backup"| M
```

O fluxo de release é **changelog-first**: o CHANGELOG é atualizado *antes* da criação da tag, de forma que o commit com tag sempre contém sua própria entrada de changelog. O fluxo tem três pontos de interação humana:

1. **Fazer merge do PR de release** — revisa o changelog e aciona a criação automática da tag
2. **Aprovar o ambiente CodeBuild** — controla o acesso às credenciais AWS para o build
3. **Publicar o draft release** — revisa os artefatos e torna o release público

`tag-on-merge.yml` dispara explicitamente `release.yml` e `codebuild.yml` via `gh workflow run --ref vX.Y.Z` após criar a tag. Os disparos são **sequenciais**: `release.yml` é executado primeiro e monitorado até a conclusão para que o draft release exista antes de `codebuild.yml` enviar os artefatos. Isso é necessário porque tags criadas com `GITHUB_TOKEN`

não disparam eventos `on: push: tags` — mas `workflow_dispatch` está isento dessa limitação. Ambos os workflows também mantêm `push: tags: v*` como fallback para pushes manuais de tags. O workflow `codebuild.yml` requer **aprovação manual** via o ambiente protegido `codebuild` antes do build prosseguir. A etapa de upload lida com todos os estados do release de forma resiliente:

- **Draft existente** (caso normal) — `release.yml` termina em ~30s criando o draft; o CodeBuild leva minutos, então o draft está pronto quando os artefatos são enviados
- **Sem release ainda** (codebuild terminou primeiro) — cria um draft com os artefatos do build; `release.yml` o atualizará depois
- **Já publicado** (nova execução) — tenta substituir os artefatos, avisa gentilmente se forem imutáveis

Estratégia de backup: Se a execução do CodeBuild disparada pela tag falhar ou for bloqueada, um administrador pode disparar manualmente o workflow via `workflow_dispatch` e selecionar a tag `v*` no seletor de branch/tag da UI do GitHub. Como `github.ref` é resolvido para a tag selecionada, a etapa de upload é ativada automaticamente.

Pipeline 2: Integração Contínua

```
flowchart LR
    A["git push main"] --> B["Aprovação manual\n(ambiente codebuild)"]
    C["workflow_dispatch\n(sem input de tag)"] --> B
    D["pull_request\n(aidlc-rules/** alterado)"] --> E["label\nrules?"]
    E -->|sim| F["label-cleanup\n(remove comentário lembrete)"]
    F --> B
    E -->|não| I["label-reminder\n(aviso + comentário no PR)"]
    B --> G["Executa AWS CodeBuild"]
    G --> H["Envia artefatos do workflow"]
```

Pipeline 3: Varredura de Segurança

```
flowchart TD
    A["push main"] --> G["security-scanners.yml"]
    B["pull_request para main"] --> G
    C["schedule (diário 03:47 UTC)"] --> G
    D["workflow_dispatch"] --> G

    G --> H["gitleaks\n(detecção de segredos)"]
    G --> I["semgrep\n(SAST multilinguagem)"]
    G --> J["grype\n(SCA de dependências)"]
    G --> K["bandit\n(SAST Python)"]
    G --> L["checkov\n(varredura de IaC)"]
    G --> M["clamav\n(varredura de malware)"]

    H --> N["Envia SARIF\npara Code Scanning"]
    I --> N
    J --> N
    K --> N
    L --> N
    M --> O["Envia log de texto\n(apenas artefato)"]
```

Todos os seis jobs de scanner são executados em paralelo. Cada scanner (exceto ClamAV) produz um relatório SARIF enviado tanto para o GitHub Code Scanning (aba Security) quanto como artefato de workflow para download. Todos os scanners usam um **padrão de falha adiada**: a varredura é executada até o final, os resultados são sempre enviados, e somente então o job falha se os achados excederem o limite configurado. Veja a referência do [Workflow de Scanners de Segurança](#) para detalhes.

Pipeline 4: Validação de Pull Request

```
flowchart TD
    A["pull_request_target\n(para main)"] --> B["get-pr-info"]
    C["merge_group\n(checks_requested)"] --> B

    B --> D["check-merge-status\n(HALT_MERGES + PRs de release abertos)"]
    B --> E["fail-by-label\n(label do-not-merge)"]
    A --> F["validate\n(título de commit convencional)"]
    A --> G["contributorStatement\n(confirmação no corpo do PR)"]
    A --> H["auto-label\n(actions/labeler)"]
```

`pull-request-lint.yml` é executado em todos os PRs direcionados a `main` e nas verificações da fila de merge. Ele impõe quatro gates (títulos de PR no formato de commit convencional, a declaração do contribuidor do modelo de PR, um mecanismo de bloqueio de merge configurável e uma verificação de label do-not-merge) e aplica labels

automaticamente com base nos caminhos dos arquivos alterados. O workflow usa `pull_request_target` (não `pull_request`) para que seja executado no contexto do branch base — isso é seguro porque nunca faz checkout nem executa código do PR, e o job `auto-label` usa `actions/labeler` que apenas lê caminhos de arquivos pela API.

Referência de Workflows

Workflow de Release PR (`release-pr.yml`)

Propriedade	Valor
Arquivo	<code>.github/workflows/release-pr.yml</code>
Gatilho	<code>workflow_dispatch</code> com input opcional de <code>version</code>
Ambiente	<i>(nenhum)</i>
Runner	<code>ubuntu-latest</code>

Propósito: Gera um `CHANGELOG.md` atualizado a partir de commits convencionais usando git-cliff, escreve a versão do release em `aidlc-rules/VERSION` e abre um PR no branch `release/vX.Y.Z`. Este é o primeiro passo no fluxo de release changelog-first. A atualização de `aidlc-rules/VERSION` garante que o PR toque em `aidlc-rules/`, o que aciona o filtro de caminho de `codebuild.yml` e o auto-label `rules`.

Job: `release-pr` ("Create Release PR")

Etapa	Nome	Ação
1	Checkout do código	<code>actions/checkout</code> com <code>fetch-depth: 0</code> (histórico completo para git-cliff)
2	Instalar git-cliff	<code>orhun/git-cliff-action</code> para disponibilizar o CLI
3	Determinar versão	Usar <code>inputs.version</code> (com validação semver) ou <code>git-cliff --bumped-version</code> para auto-deteção; fallback para bump de patch a partir da última tag
4	Verificar se a tag não existe	Falhar cedo se a tag alvo já existir
5	Gerar changelog	<code>orhun/git-cliff-action</code> com <code>--tag vX.Y.Z</code> para gerar <code>CHANGELOG.md</code>
6	Criar PR de release	Escrever versão em <code>aidlc-rules/VERSION</code> , verificar se o branch não existe, fazer commit, push do branch <code>release/vX.Y.Z</code> e abrir PR (com labels <code>release</code> e <code>rules</code> se existirem)

Deteção de versão: Se uma versão for especificada, deve ser semver válido (`MAJOR.MINOR.PATCH`); tanto `v0.2.0` quanto `0.2.0` são aceitos. Se nenhuma versão for especificada, `git-cliff --bumped-version` determina a próxima versão a partir dos prefixos de commit convencional. A configuração `[bump]` em `cliff.toml` controla as regras (ex.: `feat` → bump minor, breaking change → bump major). Se nenhum commit convencional for encontrado, o workflow faz fallback para um bump de patch a partir da última tag. Se não houver tags, ele termina com um aviso (nenhum PR é criado).

Ações externas (fixadas por SHA):

Ação	Versão	SHA
<code>actions/checkout</code>	v6.0.1	<code>8e8c483db84b4bee98b60c0593521ed34d9990e8</code>
<code>orhun/git-cliff-action</code>	v4.7.0	<code>e16f179f0be49ecdfe63753837f20b9531642772</code>

Workflow de Tag Release (`tag-on-merge.yml`)

Propriedade	Valor
Arquivo	<code>.github/workflows/tag-on-merge.yml</code>
Gatilho	<code>pull_request: types: [closed]</code>
Condição	PR foi mergeado E nome do branch começa com <code>release/v</code>
Ambiente	<i>(nenhum)</i>
Runner	<code>ubuntu-latest</code>

Propósito: Cria automaticamente uma tag de versão no commit de merge quando um PR de release é mergeado, depois dispara `release.yml` (aguarda conclusão) seguido de `codebuild.yml`.

Job: `tag` ("Create Release Tag")

Etapas	Nome	Ação
1	Criar tag	Extraí versão do nome do branch, verifica se a tag não existe, cria via GitHub API
2	Disparar workflow de release e aguardar	<code>gh workflow run release.yml --ref \$TAG --repo \$REPO</code> , depois <code>gh run watch</code> até conclusão
3	Disparar workflow do codebuild	<code>gh workflow run codebuild.yml --ref \$TAG --repo \$REPO</code> (executa após o draft release existir)

Criação de tag: Usa `gh api repos/.../git/refs` para criar uma tag leve.

Dispatch de workflow: Tags criadas com `GITHUB_TOKEN` não disparam eventos `on: push: tags` em outros workflows. Para contornar isso, `tag-on-merge.yml` dispara explicitamente `release.yml` e `codebuild.yml` via `gh workflow run --ref $TAG`. O evento `workflow_dispatch` é isento dessa limitação do `GITHUB_TOKEN`. Como `--ref` é definido como a tag, ambos os workflows disparados veem `github.ref = refs/tags/vX.Y.Z` — idêntico a um push real de tag. Os disparos são **sequenciais**: `release.yml` é executado primeiro (monitorado via `gh run watch`) para garantir que o draft release exista antes de `codebuild.yml` tentar enviar artefatos. Se a execução do release não puder ser encontrada ou falhar, `codebuild.yml` é disparado mesmo assim como fallback.

Segurança: O nome do branch `release/vX.Y.Z` é passado por uma variável de ambiente (não interpolado diretamente) para prevenir injeção de comando. A condição `if` no nível do job usa `github.event.pull_request.merged == true` para garantir que apenas PRs mergeados acionem a criação de tags.

Workflow do CodeBuild (`codebuild.yml`)

Propriedade	Valor
Arquivo	<code>.github/workflows/codebuild.yml</code>
Gatilhos	<code>push</code> para <code>main</code> , <code>push</code> de tags <code>v*</code> , <code>pull_request</code> para <code>main</code> (com gate de label, filtro de caminho), <code>workflow_dispatch</code> (disparado por <code>tag-on-merge.yml</code> ou manual — selecione uma tag na UI para acionar um build de release)
Ambiente	<code>codebuild</code> (protegido, aprovação manual)
Runner	<code>ubuntu-latest</code>
Concorrência	Agrupado por <code>{workflow}-{event_name}-{ref}</code> , cancela em progresso

Propósito: Executa um projeto AWS CodeBuild, baixa artefatos primários e secundários do S3, os armazena em cache no GitHub Actions, os envia como artefatos de workflow e (quando acionado por uma tag `v*`) os anexa ao GitHub Release.

Gate de label do PR: Para eventos `pull_request`, o workflow só é acionado quando arquivos em `aidlc-rules/**` são alterados (via filtro `paths`) e o job `build` só é executado quando a label `rules` está presente no PR (via `contains(github.event.pull_request.labels.*.name, 'rules')`). A label `rules` é aplicada automaticamente pelo job `auto-label` em `pull-request-lint.yml` (veja [Workflow de Validação de Pull Request](#)). O gatilho inclui `types: [opened, synchronize, reopened, labeled]` para que pushes subsequentes a um PR com label re-acionem o build automaticamente. Eventos `push`, `workflow_dispatch` e tags ignoram a verificação de label.

Job: `label-reminder` (somente PR, sem label `rules`)

Etapa	Nome	Ação
1	Avisar sobre label rules ausente	Emite uma anotação <code>::warning::</code> visível no resumo do Actions
2	Comentar no PR	Posta um comentário único no PR (idempotente — ignora se o comentário lembrete já existir)

Este job é executado apenas para eventos `pull_request` onde `aidlc-rules/**` foi alterado mas a label `rules` está ausente. Alerta mantenedores e revisores de que o pipeline de avaliação não foi acionado. O comentário é postado uma vez por PR usando um marcador de comentário HTML (`<!-- rules-label-reminder -->`) para evitar duplicatas. Em operação normal, o job `auto-label` em `pull-request-lint.yml` aplica a label `rules` automaticamente, de modo que este job serve como uma rede de segurança de fallback.

Job: `label-cleanup` (somente PR, label `rules` presente)

Etapa	Nome	Ação
1	Remover comentário lembrete	Encontra e exclui o comentário PR do <code>label-reminder</code> (sem operação se não existir)

Este job é executado quando a label `rules` é aplicada, removendo imediatamente o comentário lembrete sem aguardar a aprovação do gate do ambiente `codebuild`.

Job: `build`

Etapa	Nome	Condição	Ação
1	Listar caches	(sempre)	<code>gh cache list</code> para caches existentes do projeto
2	Verificar cache	(sempre)	<code>actions/cache/restore</code> com <code>lookup-only: true</code>
3	Configurar credenciais AWS	cache miss	<code>aws-actions/configure-aws-credentials</code> (OIDC)
4	Executar CodeBuild	cache miss	<code>aws-actions/aws-codebuild-run-build</code> com buildspec inline
5	Build ID	cache miss (sempre)	Echo do ID do build CodeBuild
6	Baixar artefatos do CodeBuild	cache miss	Baixar artefatos primários + secundários do S3
7	Listar artefatos do CodeBuild	cache miss	Listar e inspecionar arquivos zip baixados
8	Limpar caches de relatórios antigos	cache miss	Excluir os 3 caches mais antigos correspondentes ao branch
9	Salvar relatório em cache	cache miss	<code>actions/cache/save</code> com chave <code>{project}-{branch}-{sha}</code>
10	Enviar artefato primário	<code>!env.ACT</code>	<code>actions/upload-artifact</code> para <code>{project}.zip</code>
11	Enviar artefato de avaliação	<code>!env.ACT</code>	<code>actions/upload-artifact</code> para <code>evaluation.zip</code>
12	Enviar artefato de tendência	<code>!env.ACT</code>	<code>actions/upload-artifact</code> para <code>trend.zip</code>
13	Enviar artefatos ao release	acionado por tag <code>v*</code>	Anexar artefatos do build ao GitHub Release (draft ou publicado)

Estratégia de cache: A chave de cache `{project}-{branch}-{sha}` garante que o mesmo commit no mesmo branch nunca seja construído duas vezes. Em caso de cache hit, as

etapas 3–9 são completamente ignoradas.

Buildspec inline: O workflow embute um `buildspec-override` completo em vez de referenciar um arquivo externo. O buildspec:

- Instala `gh` CLI (via `dnf`) e `uv` (gerenciador de pacotes Python)
- Determina o contexto do build: release (com tag), pré-release (branch padrão) ou pré-merge (branch de feature)
- Cria arquivos de placeholder de avaliação e relatório de tendência em `.codebuild/`
- Produz um artefato primário (todos os arquivos em `.codebuild/`) e dois artefatos secundários (`evaluation`, `trend`)

Compatibilidade de upload de artefatos: As etapas de upload são condicionadas por `!env.ACT` porque `actions/upload-artifact` v6 é incompatível com o runner local `act`.

Ações externas (todas fixadas por SHA):

Ação	Versão	SHA
<code>actions/cache/restore</code>	v5.0.3	<code>cdf6c1fa76f9f475f3d7449005a359c84ca0f306</code>
<code>aws-actions/configure-aws-credentials</code>	v6.0.0	<code>8df5847569e6427dd6c4fb1cf565c83acfa8afa7</code>
<code>aws-actions/aws-codebuild-run-build</code>	v1.0.18	<code>d8279f349f3b1b84e834c30e47c20dcb888b7e5</code>
<code>actions/cache/save</code>	v5.0.3	<code>cdf6c1fa76f9f475f3d7449005a359c84ca0f306</code>
<code>actions/upload-artifact</code>	v6.0.0	<code>b7c566a772e6b6bfb58ed0dc250532a479d7789f</code>

Workflow de Release (`release.yml`)

Propriedade	Valor
Arquivo	<code>.github/workflows/release.yml</code>
Gatilhos	<code>workflow_dispatch</code> (disparado por <code>tag-on-merge.yml</code>), <code>push</code> em tags correspondendo a <code>v*</code> (fallback para pushes manuais de tags)
Ambiente	<i>(nenhum)</i>
Runner	<code>ubuntu-latest</code>

Propósito: Cria um **draft** de GitHub Release com um zip de `aidlc-rules/` quando disparado ou quando uma tag de versão é enviada. O release é mantido como draft para que os artefatos do CodeBuild possam ser anexados e revisados antes da publicação.

Job: `release` ("Create Release")

Etapa	Nome	Condição	Ação
1	Checkout do código	<i>(sempre)</i>	<code>actions/checkout</code> com <code>fetch-depth: 0</code>
2	Extraír versão	<i>(sempre)</i>	Guard: se <code>GITHUB_REF</code> não for uma tag <code>v*</code> , emite <code>::warning::</code> e ignora as etapas restantes. Caso contrário, analisa em <code>version</code> (sem <code>v</code>) e <code>tag</code> (com <code>v</code>)
3	Criar artefato de release	ref é uma tag <code>v*</code>	<code>zip -r ai-dlc-rules-v{VERSION}.zip aidlc-rules/</code>
4	Criar GitHub Release	ref é uma tag <code>v*</code>	<code>softprops/action-gh-release</code> com <code>draft: true</code> e zip anexado

Ignorar graciosamente: Se disparado a partir de um branch em vez de uma tag (ex.: alguém executa manualmente o workflow a partir de `main`), o job é concluído com sucesso com uma anotação de aviso em vez de falhar. Isso evita falhas confusas com X vermelho na UI do Actions.

Nomenclatura do release: `AI-DLC Workflow v{VERSION}` (ex.: `AI-DLC Workflow v0.1.6`)

Ações externas (fixadas por SHA):

Ação	Versão	SHA
<code>actions/checkout</code>	v6.0.1	<code>8e8c483db84b4bee98b60c0593521ed34d9990e8</code>
<code>softprops/action-gh-release</code>	v2.5.0	<code>a06a81a03ee405af7f2048a818ed3f03bbf83c7b</code>

Workflow de Validação de Pull Request (`pull-request-lint.yml`)

Propriedade	Valor
Arquivo	<code>.github/workflows/pull-request-lint.yml</code>
Gatilhos	<code>pull_request_target</code> para <code>main</code> (edited, labeled, opened, ready_for_review, reopened, synchronize, unlabeled); <code>merge_group</code> (checks_requested)
Ambiente	(nenhum)
Runner	<code>ubuntu-latest</code>
Concorrência	Agrupado por <code>{workflow}-{event_name}-{ref}</code> , cancela em progresso

Propósito: Valida pull requests antes do merge. Impõe títulos de PR no formato de commit convencional, a declaração de reconhecimento do contribuidor, controles de bloqueio de merge e um gate de label do-not-merge. Também é executado como verificação da fila de merge.

Por que `pull_request_target`: Este gatilho executa o workflow no contexto do branch base (não do head do PR). Isso é seguro aqui porque nenhuma etapa faz checkout ou executa código do PR — o workflow apenas inspeciona metadados do PR (título, labels, corpo). Usar `pull_request_target` garante que o workflow tenha acesso a segredos e labels do repositório mesmo para PRs de forks.

Job: `get-pr-info`

Etapa	Nome	Ação
1	Obter info do PR	Extraí número do PR e labels do contexto do evento (<code>pull_request_target</code>) ou via lookup de API (<code>merge_group</code>)

Produz `pr_number` e `pr_labels` para jobs downstream. Para eventos `merge_group`, o número do PR é extraído do nome do ref e as labels são obtidas via GitHub API. Para eventos `pull_request_target`, os valores vêm diretamente do payload do evento.

Job: `check-merge-status` ("Check Merge Status")

Depende de `get-pr-info`. Executa com `if: always()` para rodar mesmo se o job upstream falhar.

Verificação	Comportamento
PRs de release abertos	Bloqueia o merge se outros PRs <code>release/</code> estiverem abertos (evita releases concorrentes)
<code>HALT_MERGES = 0</code>	Todos os merges permitidos (padrão)
<code>HALT_MERGES = -N</code>	Todos os merges bloqueados
<code>HALT_MERGES = N</code>	Apenas o PR #N tem permissão de merge

Job: `fail-by-label` ("Fail by Label")

Depende de `get-pr-info`. Executa com `if: always()`. Falha na verificação se o PR tem a label `do-not-merge` (configurável via variável `DO_NOT_MERGE_LABEL`).

Job: `validate` ("Validate PR title")

Executado apenas para eventos `pull_request` e `pull_request_target` (não `merge_group`).

Usa `amannn/action-semantic-pull-request` para impor o formato de commit convencional nos títulos dos PRs.

Tipos permitidos: `fix`, `feat`, `build`, `chore`, `ci`, `docs`, `style`, `refactor`, `perf`, `test`.

Escopos são opcionais (`requireScope: false`).

Job: `auto-label` ("Auto-label")

Executado apenas para eventos `pull_request_target`. Usa `actions/labeler` v6.0.1 para aplicar e remover labels automaticamente com base nos caminhos dos arquivos alterados. As regras de label são definidas em `.github/labeler.yml`:

Label	Padrão de Caminho	Descrição
<code>rules</code>	<code>aidlc-rules/**</code>	Aciona o pipeline de avaliação do CodeBuild
<code>documentation</code>	<code>**/*.md</code> (excluindo <code>aidlc-rules/**</code>)	Alterações em arquivos markdown que não são regras
<code>github</code>	<code>.github/**</code>	Alterações em workflows, templates ou configs

Com `sync-labels: true`, as labels são removidas automaticamente quando os arquivos correspondentes não estão mais no diff do PR (ex.: após um rebase que descarta essas alterações). Novas regras de label podem ser adicionadas editando `.github/labeler.yml` — nenhuma alteração no workflow é necessária.

Job: `contributorStatement` ("Require Contributor Statement")

Executado apenas para eventos `pull_request` e `pull_request_target`. Ignorado para contas de bot (`dependabot[bot]`, `github-actions[bot]`, `github-actions`, `aidlc-workflows`). Verifica se o corpo do PR contém o texto de reconhecimento do contribuidor de

`.github/pull_request_template.md`:

Ao submeter este pull request, confirmo que você pode usar, modificar, copiar e redistribuir esta contribuição, sob os termos da licença do projeto.

Ações externas (fixadas por SHA):

Ação	Versão	SHA
<code>actions/labeler</code>	v6.0.1	<code>634933edcd8ababfe52f92936142cc22ac488b1b</code>
<code>amannn/action-semantic-pull-request</code>	v6.1.1	<code>48f256284bd46cdaab1048c3721360e808335d50</code>
<code>actions/github-script</code>	v8.0.0	<code>ed597411d8f924073f98dfc5c65a23a2325f34cd</code>

Workflow de Scanners de Segurança (`security-scanners.yml`)

Propriedade	Valor
Arquivo	<code>.github/workflows/security-scanners.yml</code>
Gatilhos	<code>push</code> para <code>main</code> , <code>pull_request</code> para <code>main</code> , <code>schedule</code> (diário 03:47 UTC), <code>workflow_dispatch</code>
Ambiente	(nenhum)
Runner	<code>ubuntu-latest</code>
Concorrência	Agrupado por <code>{workflow}-{event_name}-{ref}</code> , cancela em progresso

Propósito: Executa seis scanners de segurança independentes em paralelo para detectar segredos, vulnerabilidades, configurações incorretas e malware. Todos os achados HIGH e CRITICAL devem ser corrigidos ou ter uma aceitação de risco documentada antes do merge (veja [Requisitos para Achados de Segurança](#)).

Modelo de permissões: Negar tudo no nível do workflow, então cada job concede apenas `actions: read`, `contents: read` e `security-events: write`.

Jobs:

Job	Scanner	O que detecta	Falha em
<code>gitleaks</code>	Gitleaks	Segredos no histórico git	Qualquer segredo fora do <code>.gitleaks-baseline.json</code>
<code>semgrep</code>	Semgrep	Padrões de segurança problemáticos (todas linguagens)	Qualquer achado (PRs: apenas novos achados via <code>--baseline-commit</code>)
<code>grype</code>	Grype	CVEs conhecidas em dependências	CVEs altas ou críticas (<code>fail-on-severity: high</code>)
<code>bandit</code>	Bandit	Problemas de segurança em Python	Qualquer achado com alta confiança
<code>checkov</code>	Checkov	Configurações incorretas de IaC (GitHub Actions, Dockerfiles)	Qualquer falha de verificação (menos as ignoradas)
<code>clamav</code>	ClamAV	Malware e vírus	Qualquer detecção

Padrão de falha adiada: Todos os scanners capturam o código de saída sem falhar na etapa (`set +e`), enviam o relatório SARIF como artefato e para o GitHub Code Scanning, depois fazem o job falhar se achados foram detectados. Isso garante que os resultados sejam sempre preservados independentemente do resultado. O ClamAV segue o mesmo padrão, mas envia um log de texto em vez de SARIF.

Arquivos de configuração:

Arquivo	Propósito
<code>.bandit</code>	Alvos, exclusões e nível de confiança do Bandit
<code>.semgrepignore</code>	Exclusões de caminho do Semgrep
<code>.gitleaks.toml</code>	Extensão de regras e allowlist de caminhos do Gitleaks
<code>.gitleaks-baseline.json</code>	Achados conhecidos pré-existent (credenciais de teste)
<code>.grype.yaml</code>	Limite de severidade e lista de CVEs ignoradas do Grype
<code>.checkov.yaml</code>	Frameworks e verificações ignoradas do Checkov

Fixação de versões: Todas as versões de ferramentas de scanner e GitHub Actions são fixadas em versões específicas ou SHAs de commit no arquivo de workflow para garantir builds reproduzíveis e prevenir ataques à cadeia de fornecimento. Essas fixações devem ser revisadas e atualizadas periodicamente (no mínimo trimestralmente). Veja [Atualizando Versões Fixadas](#) para o procedimento de atualização.

Para instruções detalhadas de correção e supressão, veja [Guia do Desenvolvedor — Scanners de Segurança](#).

Ambientes Protegidos

Ambiente	Usado Por	Propósito
<code>codebuild</code>	<code>codebuild.yml</code> job <code>build</code>	Controla o acesso às credenciais AWS para CodeBuild

O ambiente `codebuild` é o único ambiente protegido. Ele contém:

- O segredo `AWS_CODEBUILD_ROLE_ARN` (necessário para assumir a role AWS via OIDC)
- Possivelmente as variáveis de repositório `CODEBUILD_PROJECT_NAME`, `AWS_REGION` e `ROLE_DURATION_SECONDS` (estas podem alternativamente ser definidas no nível do repositório)

As regras de proteção do ambiente (configuradas nas configurações do repositório GitHub) podem incluir revisores obrigatórios ou restrições de branch de deploy.

Segredos e Variáveis

Segredos

Segredo	Escopo	Usado Por	Propósito
<code>AWS_CODEBUILD_ROLE_ARN</code>	Ambiente (<code>codebuild</code>)	<code>codebuild.yml</code>	ARN da Role IAM para assumir role AWS STS via OIDC
<code>GITHUB_TOKEN</code>	Automático (fornecido pelo GitHub)	<code>release.yml</code> , <code>release-pr.yml</code> , <code>tag-on-merge.yml</code> , <code>pull-request-lint.yml</code>	Autenticar chamadas à API do GitHub (criação de release, PR, tag, dispatch de workflow, validação de PR)

O workflow `codebuild.yml` também usa `github.token` (o token automático, acessado sem o prefixo `secrets.`) para gerenciamento de cache e uploads de assets de release.

Variáveis de Repositório

Variável	Usado Por	Fallback Padrão	Propósito
<code>CODEBUILD_PROJECT_NAME</code>	<code>codebuild.yml</code>	<code>codebuild-project</code>	Nome do projeto AWS CodeBuild
<code>AWS_REGION</code>	<code>codebuild.yml</code>	<code>us-east-1</code>	Região AWS para CodeBuild e STS
<code>ROLE_DURATION_SECONDS</code>	<code>codebuild.yml</code>	<code>7200</code>	Duração da sessão STS (segundos)
<code>DO_NOT_MERGE_LABEL</code>	<code>pull-request-lint.yml</code>	<code>do-not-merge</code>	Nome da label que bloqueia o merge do PR
<code>HALT_MERGES</code>	<code>pull-request-lint.yml</code>	<code>0</code>	Gate de merge: <code>0</code> = permitir todos, <code>-N</code> = bloquear todos, <code>N</code> = apenas PR #N

Todas as variáveis têm defaults sensatos via sintaxe `${{ vars.VAR || 'default' }}`, de modo que os workflows funcionam mesmo sem configuração explícita de variáveis.

Modelo de Permissões

Permissões no nível do workflow

Workflow	Permissões
<code>codebuild.yml</code>	Todos os 16 escopos explicitamente definidos como <code>none</code>
<code>pull-request-lint.yml</code>	Todos os 16 escopos explicitamente definidos como <code>none</code>
<code>release.yml</code>	Todos os 16 escopos explicitamente definidos como <code>none</code>
<code>release-pr.yml</code>	Todos os 16 escopos explicitamente definidos como <code>none</code>
<code>security-scanners.yml</code>	Todos os 16 escopos explicitamente definidos como <code>none</code>
<code>tag-on-merge.yml</code>	Todos os 16 escopos explicitamente definidos como <code>none</code>

Permissões no nível do job (substituições)

Workflow	Job	Permissões	Justificativa
<code>codebuild.yml</code>	<code>label-reminder</code>	<code>pull-requests:</code> <code>write</code>	Postar comentário lembrete quando label <code>rules</code> está ausente
<code>codebuild.yml</code>	<code>label-cleanup</code>	<code>pull-requests:</code> <code>write</code>	Excluir comentário lembrete quando label <code>rules</code> é aplicada
<code>codebuild.yml</code>	<code>build</code>	<code>actions: write</code> , <code>contents: write</code> , <code>id-token: write</code>	Gerenciamento de cache, upload de assets de release, token OIDC para AWS STS
<code>pull-request-lint.yml</code>	<code>auto-label</code>	<code>contents: read</code> , <code>issues: write</code> , <code>pull-requests:</code> <code>write</code>	Aplicar/remover labels com base nos caminhos dos arquivos; <code>issues: write</code> permite criar labels que ainda não existem
<code>pull-request-lint.yml</code>	<code>get-pr-info</code>	<code>contents: read</code> , <code>pull-requests:</code> <code>read</code>	Ler metadados e labels do PR via API
<code>pull-request-lint.yml</code>	<code>check-merge-status</code>	<code>pull-requests:</code> <code>read</code>	Ler estado do PR para verificações de gate de merge
<code>pull-request-lint.yml</code>	<code>validate</code>	<code>pull-requests:</code> <code>read</code>	Ler título do PR para validação de commit convencional
<code>pull-request-lint.yml</code>	<code>contributorStatement</code>	<code>pull-requests:</code> <code>read</code>	Ler corpo do PR para reconhecimento do contribuidor
<code>release.yml</code>	<code>release</code>	<code>contents: write</code>	Criar draft release e anexar artefato zip

Workflow	Job	Permissões	Justificativa
<code>release-pr.yml</code>	<code>release-pr</code>	<code>contents: write</code> , <code>pull-requests: write</code>	Gerar changelog, fazer push do branch, abrir PR
<code>tag-on-merge.yml</code>	<code>tag</code>	<code>contents: write</code> , <code>actions: write</code>	Criar tag via API, disparar workflows de release e codebuild

Todos os seis workflows seguem um padrão **negar-tudo-depois-conceder**: cada escopo de permissão é definido como `none` no nível do workflow, então apenas os escopos necessários são concedidos no nível do job. Esta é a configuração mais restritiva possível e evita escalonamento de privilégios por etapas comprometidas. `security-scanners.yml` concede a cada um dos seus seis jobs `actions: read`, `contents: read` e `security-events: write`.

Postura de Segurança

Controle	Implementação
Proteção da cadeia de fornecimento	Todas as ações externas fixadas em SHAs completos de commit (não em tags de versão mutáveis)
Autenticação AWS	Assumir role baseada em OIDC via <code>id-token: write</code> — sem credenciais estáticas armazenadas
Tokens com privilégio mínimo	Todos os seis workflows negam explicitamente todos os 16 escopos de permissão no nível do workflow, concedendo apenas os escopos necessários no nível do job
Proteção de ambiente	O ambiente <code>codebuild</code> controla o acesso a credenciais AWS com possíveis regras de revisor/branch
Varredura de segurança	Seis scanners automatizados (SAST, SCA, segredos, IaC, malware) executados em cada push para <code>main</code> , cada PR e diariamente. Achados são publicados no GitHub Code Scanning. Todos os achados HIGH e CRITICAL requerem correção ou aceitação documentada de risco
CI com gate de label	<code>codebuild.yml</code> requer a label <code>rules</code> em PRs e só é acionado para alterações em <code>aidlc-rules/**</code> , evitando builds desnecessários e prompts de aprovação de ambiente. A label é aplicada automaticamente pelo job <code>auto-label</code> em <code>pull-request-lint.yml</code>
Controle de concorrência	<code>codebuild.yml</code> , <code>pull-request-lint.yml</code> e <code>security-scanners.yml</code> cancelam execuções em andamento para o mesmo branch
Gatilho seguro de PR	<code>pull-request-lint.yml</code> usa <code>pull_request_target</code> mas nunca faz checkout do código do PR — apenas inspeciona metadados (título, labels, corpo)
Inputs seguros contra injeção	Zero interpolação de expressão <code>\${{ }}</code> em blocos <code>run:</code> — todos os valores dinâmicos (<code>github.ref_name</code> , <code>github.repository</code> , <code>env.*</code> , inputs de evento) passados via <code>env:</code> de etapa ou variáveis de ambiente <code>env:</code> automáticas do workflow

Controle	Implementação
Propriedade de código	<code>.github/</code> (incluindo workflows) de propriedade exclusiva de <code>@awslabs/aidlc-admins</code> via CODEOWNERS
Mascaramento de conta	<code>mask-aws-account-id: true</code> na configuração de credenciais AWS

Requisitos para Achados de Segurança

Todos os achados de segurança **HIGH** e **CRITICAL** de qualquer scanner devem ser **corrigidos** ou ter uma **aceitação de risco documentada** antes que um PR possa ser mergeado para `main`. Isso se aplica a:

- **Bandit / Semgrep (SAST):** Achados de alta severidade devem ser corrigidos ou suprimidos com um comentário inline (`# nosec` / `# nosemgrep`) que inclui uma justificativa explicando por que o achado é aceitável
- **Grype (SCA):** CVEs altas e críticas devem ser resolvidas atualizando a dependência afetada. Se nenhuma correção estiver disponível, adicione uma entrada ao `ignore` do `.grype.yaml` com o CVE, o pacote afetado e o motivo de aceitação
- **Gitleaks (Segredos):** Qualquer segredo detectado deve ser rotacionado imediatamente. Apenas credenciais sintéticas/de teste podem ser adicionadas ao baseline (`.gitleaks-baseline.json`)
- **Checkov (IaC):** Verificações com falha devem ser corrigidas ou suprimidas com um comentário inline `# checkov:skip=` com um motivo, ou adicionadas ao `skip-check` do `.checkov.yaml` com um comentário
- **ClamAV (Malware):** Qualquer detecção deve ser investigada e o arquivo removido. Não existe mecanismo de supressão

Processo de aceitação de risco:

1. O desenvolvedor adiciona a supressão adequada (comentário inline ou entrada de configuração) com uma justificativa clara
2. A supressão é revisada como parte do processo normal de code review do PR
3. Revisores de `@awslabs/aidlc-admins` ou `@awslabs/aidlc-maintainers` devem aprovar qualquer aceitação de risco
4. Achados LOW e MEDIUM devem ser resolvidos quando possível, mas não bloqueiam o merge

Para instruções detalhadas de correção e supressão por scanner, veja [Guia do Desenvolvedor — Scanners de Segurança](#).

Propriedade de Código

Definida em `.github/CODEOWNERS`:

Caminho	Proprietários
<code>*</code> (padrão)	<code>@awslabs/aidlc-admins</code> <code>@awslabs/aidlc-maintainers</code>
<code>.github/</code>	<code>@awslabs/aidlc-admins</code>
<code>.github/CODEOWNERS</code>	<code>@awslabs/aidlc-admins</code>
<code>aidlc-rules/</code>	<code>@awslabs/aidlc-admins</code> <code>@awslabs/aidlc-maintainers</code> <code>@awslabs/aidlc-writers</code>
<code>assets/</code>	<code>@awslabs/aidlc-admins</code> <code>@awslabs/aidlc-maintainers</code> <code>@awslabs/aidlc-writers</code>
<code>scripts/</code>	<code>@awslabs/aidlc-admins</code> <code>@awslabs/aidlc-maintainers</code>
<code>CHANGELOG.md</code> , <code>cliff.toml</code> , <code>LICENSE</code> , etc.	<code>@awslabs/aidlc-admins</code>

Implicação principal: Apenas `@awslabs/aidlc-admins` pode aprovar alterações em `.github/` (workflows, CODEOWNERS, templates de issue).

Processo de Release

Os releases seguem um fluxo **changelog-first**: o CHANGELOG é atualizado *antes* da criação da tag, de forma que o commit com tag sempre contém sua própria entrada de changelog. O processo tem três pontos de interação humana (fazer merge do PR, aprovar o CodeBuild, publicar o release).

1. **Disparar o workflow de Release PR** via a UI do GitHub Actions:
2. Navegue até Actions → Release PR → Run workflow

3. Opcionalmente especifique uma versão (ex.: `0.2.0`); deixe em branco para auto-determinar a partir dos commits convencionais
4. `release-pr.yml` gera `CHANGELOG.md`, escreve a versão em `aidlc-rules/VERSION` e abre um PR no branch `release/v1.2.0` com as labels `release` e `rules`

5. Revisar e fazer merge do PR de release:

6. Verifique se o conteúdo do changelog está correto
7. Faça o merge do PR (requer aprovação de `@awslabs/aidlc-admins` já que `CHANGELOG.md` é de propriedade deles)
8. `tag-on-merge.yml` cria automaticamente a tag `v1.2.0` no commit de merge e dispara os workflows de release e build
9. `release.yml` **é executado automaticamente** (disparado por `tag-on-merge.yml` com `--ref v1.2.0`):
10. Compacta `aidlc-rules/` em `ai-dlc-rules-v1.2.0.zip`
11. Cria um **draft** de GitHub Release chamado "AI-DLC Workflow v1.2.0" com o zip anexado
12. `codebuild.yml` **é executado automaticamente** (disparado por `tag-on-merge.yml`; requer aprovação do ambiente `codebuild`):
13. Executa CodeBuild no commit com tag
14. Baixa artefatos do build (primário, avaliação, tendência)
15. Anexa artefatos ao draft release (ou cria um draft se ainda não existir)
16. **Publicar o release** clicando em "Publish release" na UI do GitHub:
17. Verifique se todos os artefatos esperados estão anexados (zip das regras + artefatos do build)
18. Revise as notas do release e edite se necessário

Nota: As regras de branch de deploy do ambiente protegido `codebuild` podem precisar ser atualizadas para permitir tags `v*` (além de `main`) para que builds acionados por tag prossigam.

Configuração do Changelog

Definida em `cliff.toml` (usada por `release-pr.yml`):

Configuração	Valor
Formato de commit	Commits convencionais (<code>feat:</code> , <code>fix:</code> , <code>docs:</code> , etc.)
Padrão de tag	<code>v[0-9].*</code>
Ordem de classificação	Mais antigos primeiro

Grupos de commits:

Prefixo	Nome do Grupo
<code>feat</code>	Funcionalidades
<code>fix</code>	Correções de Bugs
<code>docs</code>	Documentação
<code>perf</code>	Performance
<code>refactor</code>	Refatoração
<code>style</code>	Estilo
<code>test</code>	Testes
<code>build</code>	CI/CD
<code>ci</code>	CI/CD
<code>chore</code>	Miscelânea

Commits filtrados:

Padrão	Ação
<code>docs: update changelog</code>	Ignorado (ruído do fluxo de release anterior)

Commits não convencionais são filtrados (`filter_unconventional = true`).

Regras de bump de versão (definidas na seção `[bump]`):

Regra	Efeito
<code>features_always_bump_minor = true</code>	Commits <code>feat:</code> acionam um bump de versão minor
<code>breaking_always_bump_major = true</code>	Breaking changes acionam um bump de versão major

Essas regras são usadas por `git-cliff --bumped-version` ao auto-determinar a próxima versão em `release-pr.yml`.

Atualizando Versões Fixadas

Todas as ferramentas de scanner, GitHub Actions e imagens de container nos arquivos de workflow são fixadas em versões específicas ou SHAs de commit. Isso previne ataques à cadeia de fornecimento e garante builds reproduzíveis, mas requer manutenção periódica para se manter atualizado com patches de segurança e novos recursos.

As versões fixadas devem ser revisadas e atualizadas **no mínimo trimestralmente**.

Lista de verificação pré-commit do agente (recomendada):

- `npx markdownlint-cli2 --fix "**/*.md" # corrige automaticamente problemas de lint no markdown`
- `npx markdownlint-cli2 "**/*.md" # verifica se não há erros de lint`
- `uv run pytest # executa os testes via uv`

Os agentes devem executar a lista de verificação acima e garantir que todas as verificações passem antes de fazer commit e push das alterações.